

Project U.S. Patent Phrase to Phrase Matching

Ivan Vasilev, Ilya Gromakov

May 2026

Abstract

This work addresses semantic phrase matching in patent documents, a task with significant practical value for patent prior art search. While prior approaches predominantly relied on Word2Vec-based embeddings or BERT-like encoder ensembles, we explore a novel direction: leveraging decoder-only large language models with prompt-based feature extraction and a learned regression head. To the best of our knowledge, this is the first investigation of decoder-only LLMs on the Patent Phrase Similarity benchmark. Code is available at https://github.com/Chipchilinka/nlp_project_uspppm/tree/main.

1 Introduction

The work is devoted to the task of semantic phrase matching in patent documents, which was the basis of the competition U.S. Patent Phrase to Phrase Matching held on the Kaggle platform. This task has significant practical value for patent offices, patent attorneys, and researchers, since during the examination procedure it is critically important to determine whether a given invention has been described earlier — that is, to conduct a so-called prior art search.

While prior approaches to this problem have predominantly relied on Word2Vec-based embeddings [Aslanyan and Wetherbee, 2022] or ensembles of BERT-like encoder models [Yu et al., 2024, Chenghuige, 2022], our work explores a fundamentally different direction: leveraging decoder-only large language models (LLMs) for semantic similarity assessment. Unlike encoder-based architectures that produce fixed-size sentence embeddings, decoder-only models offer the potential for more flexible, context-aware representations. We combine prompt-based feature extraction with a learned regression head, moving beyond pure generative prompting to adapt these representations specifically for similarity scoring. To the best of our knowledge, this is the first investigation of decoder-only LLMs on the Patent Phrase Similarity benchmark, addressing the open question of whether their representational advantages translate to improved performance on domain-specific technical phrase matching.

1.1 Team

Ivan Vasilev prepared this document, refactored code.

Ilya Gromakov wrote learning code.

2 Related Work

[Aslanyan and Wetherbee, 2022] Introduced the Patent Phrase Similarity dataset (48,548 pairs, 75/5/20 split) with CPC context and baselines (e.g., GloVe Pearson 0.44). This work established the official benchmark.

[Jeremy, 2022] One approach to the competition using DeBERTa-v3-small as the backbone model. The key focus is on rapid experimentation through an effective anchor-based validation split, where 25% of unique anchors are held out to mimic the hidden test distribution. Inputs are formatted as `[section] context [s] anchor [s] target`, with patent sections added as special tokens to the vocabulary. The model is trained for 4 epochs with a cosine learning rate schedule, warmup ratio of 0.1, batch size of 128, and learning rate of $8e-5$ using fp16 precision. The approach achieves a Pearson correlation of 0.820 on the validation set, with progressive improvements from adding custom separators (0.819) and section special tokens (0.820). Further directions suggested include using patent-pretrained models, sentence similarity models, and replacing CPC codes with textual descriptions.

[Yu et al., 2024] An ensemble of four BERT-based models (DeBERTa-v3, PatentBERT, ELECTRA, etc.) with weighted averaging and custom CPC-context input pre-training. Achieved a high CV Pearson of 0.8534; computationally intensive, and code is not public.

[Chenghuige, 2022] The top competition solution (Private Board Pearson 0.8782). The core architecture uses DeBERTa-v3-large with a BiLSTM header and linear attention pooling, while freezing the BERT embedding layer to avoid excessive fine-tuning. The key innovation is two "targets grouping" strategies—grouping by [anchor, context] for the best single model and grouping by [anchor, sector] for diversity—formatted as `anchor[SEP]target[SEP]CPC-TEXT[SEP]targets` with random shuffling of targets during training. Training employed Pearson loss, Adversarial Weight Perturbation from epoch 2, and differential learning rates ($2e-5$ for BERT, $1e-3$ for the head). The final ensemble combined 4 diverse backbones (DeBERTa-v3-large weight 1, bert-for-patents 0.4, ELECTRA-large 0.3, simcse-bert-for-patent 0.2) using 5 folds + 1 full-train model (weight $\times 2$) with min-max scaling before weighted averaging, achieving progressive gains from 0.8745 (single model) to 0.8779 (with LSTM) to 0.8782 (with sector-based grouping).

написать еще про бейзлайн

3 Model Description

Our solution addresses the semantic similarity problem in patent phrase matching using a parameter-efficient fine-tuning strategy built on top of a large language model. The core idea is to combine the strong semantic understanding ca-

pabilities of instruction-tuned LLMs with domain-specific adaptation to patent terminology and CPC contexts.

3.1 Motivation

Traditional transformer encoders such as DeBERTa achieve strong results on semantic similarity tasks, but patent language introduces several additional challenges:

- highly technical vocabulary,
- domain-specific abbreviations,
- context-dependent semantics,

For example, the semantic relationship between “strong material” and “steel” depends heavily on the CPC context. General-purpose embeddings often fail to capture such specialized meaning.

To address this issue, we use:

1. Context-aware preprocessing,
2. Domain adaptation on patent-related corpora,
3. LoRA fine-tuning of a Qwen2.5-7B-Instruct model,
4. Regression-based semantic similarity prediction.

3.2 Model Architecture

The backbone model is the 7B Instruct version of Qwen2.5.

Instead of full fine-tuning, we apply Low-Rank Adaptation (LoRA).

This dramatically reduces the number of trainable parameters while enabling efficient specialization to patent language.

The prediction head performs scalar regression:

$$\hat{y} \in [0, 1]$$

where:

- 0 corresponds to unrelated phrases,
- 1 corresponds to nearly identical meaning.

3.3 Input Representation

Each training sample consists of:

- anchor phrase,
- target phrase,
- CPC context.

We concatenate them into a structured prompt format:

[CONTEXT] + [ANCHOR] + [TARGET]

Example:

Context: H04W

Anchor: mobile terminal

Target: cellular device

The CPC code provides important domain information that helps the model disambiguate terminology across technical fields.

3.4 Context Preprocessing and Domain Adaptation

Instead of using raw CPC codes directly, we enrich them using external CPC hierarchy information:

- CPC section,
- subclass descriptions,
- technological domain labels.

For example:

H04W -> Wireless communication networks

This allows the model to learn semantic relationships in a more interpretable technical space.

3.5 LoRA Fine-Tuning

To efficiently adapt the large model, we use LoRA adapters inserted into the transformer attention layers.

The trainable update is represented as:

$$W' = W + BA$$

where:

- W is the frozen pretrained weight matrix,
- A and B are low-rank trainable matrices.

This significantly reduces memory requirements while preserving the pre-trained knowledge of the base model.

The approach provides several advantages:

- lower VRAM usage,
- faster training,
- reduced overfitting risk,

3.6 Training Objective

The task is formulated as a regression problem.

The model predicts a continuous similarity score:

$$\hat{y} = f(x_a, x_t, c)$$

where:

- x_a — anchor phrase,
- x_t — target phrase,
- c — CPC context.

The optimization objective minimizes Binary Cross-Entropy with Logits Loss:

$$L = -\frac{1}{N} \sum_{i=1}^N [y_i \log \sigma(\hat{y}_i) + (1 - y_i) \log(1 - \sigma(\hat{y}_i))]$$

where:

- y_i is the ground-truth similarity score,
- $\hat{y}_i$ is the raw model output (logit),
- $\sigma(\cdot)$ denotes the sigmoid function.

Evaluation is performed using Pearson correlation, which aligns with the competition metric.

It is important that in order to obtain values, sigmoid is hung over the logs of the model.

3.7 Validation Strategy

To avoid data leakage, validation is performed using anchor-based splitting.

Samples sharing the same anchor phrase are kept within the same partition.

This setup better reflects the hidden test distribution and prevents overly optimistic validation scores.

4 Dataset

We use the U.S. Patent Phrase-to-Phrase Matching dataset, introduced as part of a public Kaggle competition organized by the United States Patent and Trademark Office (USPTO) [Aslanyan and Wetherbee, 2022]. The dataset is made freely available under the CC0 Public Domain license, ensuring it can be used for unrestricted research purposes. It can be downloaded from the official competition page at <https://kaggle.com/competitions/us-patent-phrase-to-phrase-matching>.

4.1 Data Description

Each entry consists of a pair of short phrases – an anchor and a target – together with a CPC classification code (version 2021.05) that provides the technical context in which their semantic similarity is judged. The ground-truth similarity s is a discrete rating on the ordered scale $\{0, 0.25, 0.5, 0.75, 1.0\}$, where

- 1.0 – very close match (often identical up to stopwords or morphology);
- 0.75 – close synonym or abbreviation;
- 0.5 – broader/narrower synonym (hypernym/hyponym);
- 0.25 – topically related but not synonymous;
- 0.0 – unrelated.

The scores were produced by one or more manual expert annotators.

4.2 Splits and Statistics

The competition provides a training set (`train.csv`) with similarity scores and a public test set (`test.csv`) without scores; the latter contains approximately 12 000 pairs and is used for final evaluation. For model development, we further split the training data into a training and a validation subset. To avoid information leakage, we performed a stratified split by anchor phrase: 25% of the unique anchors were randomly assigned to the validation set, and all rows containing those anchors were moved accordingly. This ensures that phrases seen during validation do not appear in the training set with opposite roles. The resulting dataset sizes and basic characteristics are summarised in Table 1.

The full list of CPC codes and their descriptive texts (e.g., A01B “agriculture; soil working...”) was obtained from the official CPC 2021.05 archive [?]. We

Table 1: Statistics of the USPTO patent phrase matching dataset. The validation split was created from the original training set using a stratified anchor-based strategy.

	Train	Validation	Test (public)
Pairs	27 352	9 118	12 000
Unique anchors	$\approx 12\,000$	$\approx 4\,000$	–
Unique targets	$\approx 10\,500$	$\approx 3\,800$	–
Unique contexts (CPC codes)	$\approx 2\,300$	$\approx 1\,900$	–

replaced the short code with its long description when forming the model input, in order to provide richer semantic context.

5 Experiments

5.1 Metrics

Following the competition setup, we evaluate model performance using the Pearson correlation coefficient between predicted and ground-truth similarity scores. This metric measures the linear correlation between two variables, producing a value in the range $[-1, 1]$, where 1 indicates perfect positive correlation, 0 indicates no linear correlation, and -1 indicates perfect negative correlation. The Pearson correlation coefficient is defined as:

$$r = \frac{\sum_{i=1}^N (y_i - \bar{y})(\hat{y}_i - \bar{\hat{y}})}{\sqrt{\sum_{i=1}^N (y_i - \bar{y})^2 \sum_{i=1}^N (\hat{y}_i - \bar{\hat{y}})^2}}$$

where:

- y_i is the ground-truth similarity score for sample i ,
- $\hat{y}_i$ is the predicted similarity score for sample i ,
- $\bar{y}$ and $\bar{\hat{y}}$ are the mean values of the ground-truth and predicted scores respectively,
- N is the total number of samples.

5.2 Experiment Setup

We conduct two main experiments to evaluate the impact of context enrichment on model performance. The first experiment uses the original CPC codes as provided in the dataset, while the second experiment replaces CPC codes with their textual descriptions to provide richer contextual information to the model. Both experiments are performed on a single NVIDIA GTX 1080Ti GPU (8GB VRAM), which constrains the batch size and training duration (each experiment lasted 35 hours).

We use `Qwen/Qwen2.5-7B-Instruct` as the base decoder-only large language model. The model is loaded with float4 precision. Input sequences are tokenized with a maximum length of 384 tokens, which balances contextual coverage with memory constraints.

Both experiments are trained for 3 epochs, with each run taking approximately 35 hours on the available hardware. We use a batch size of 4 for both training and validation, with gradient accumulation over 4 steps, resulting in an effective batch size of 16. Optimization is performed using the AdamW optimizer with a learning rate of 2×10^{-4} , weight decay of 0.01, and a cosine annealing with warmup schedule covering 10% of the total training steps. The random seed is fixed to 42 across all experiments to ensure reproducibility.

Hyperparameter	Value
Model	Qwen/Qwen2.5-7B-Instruct
Max sequence length	384
Training batch size	4
Validation batch size	4
Gradient accumulation steps	4
Effective batch size	16
Epochs	3
Learning rate	2×10^{-4}
Weight decay	0.01
Warmup ratio	0.1
Random seed	42
Precision	float16

Table 2: Hyperparameters used in both experimental runs.

We follow the standard train-validation split provided in the competition dataset (`train.csv`), using the designated training portion for model fine-tuning and the validation portion for hyperparameter selection. To avoid data leakage, validation is performed using anchor-based splitting: samples sharing the same anchor phrase are kept within the same partition. This setup better reflects the hidden test distribution and prevents overly optimistic validation scores that could arise if phrases with identical anchors were distributed across both training and validation sets.

6 Results

Model	Source	Pearson
DeBERTa-v3-small	[Jeremy, 2022]	0.820
DeBERTa-v3-large	[Yu et al., 2024]	0.851
Ensemble (4 models)	[Yu et al., 2024]	0.852
DeBERTa-v3-large*	[Chenghuige, 2022]	0.860
Ensemble (7 models)	[Chenghuige, 2022]	0.878
Our LLM + regression head	This work	TODO (CV 85.6)

Table 3: Comparison of results on the Patent Phrase Similarity benchmark.

Several notable observations emerge from the results in Table 3. First, the gap between DeBERTa-v3-large (0.851) and DeBERTa-v3-large* (0.860) is striking given that both use the same backbone architecture—the difference stems primarily from prompt engineering and input formatting. The latter employs a more sophisticated prompt design with section-based special tokens and grouped target formatting, demonstrating that careful prompt construction can yield improvements comparable to or even exceeding those obtained by ensembling multiple models. Indeed, the 4-model ensemble in [Yu et al., 2024] achieves only a marginal gain over its single best backbone (0.852 vs. 0.851), suggesting that ensemble diversity benefits may be limited when individual models share similar prompting strategies.

Our decoder-only LLM with a regression head achieves a cross-validation Pearson score of 0.856, placing it between the standard DeBERTa-v3-large and the prompt-optimized DeBERTa-v3-large*. Notably, our current prompt design is more similar to the simpler approach in [Yu et al., 2024], leaving substantial room for improvement through more advanced prompt engineering—for instance, incorporating CPC textual descriptions or section-based special tokens. This suggests that with better prompt design, our approach could potentially surpass the DeBERTa-v3-large* baseline. Finally, the 7-model ensemble in [Chenghuige, 2022] leads by a significant margin (0.878), combining diverse backbones with different weighting strategies, which highlights the effectiveness of leveraging both architectural diversity and prompt diversity in ensemble construction.

7 Conclusion

In this work, we explored the use of decoder-only large language models for semantic phrase matching in patent documents, addressing the U.S. Patent Phrase to Phrase Matching task. We proposed an approach that combines prompt-based feature extraction from Qwen2.5-7B-Instruct with a learned regression head, adapting decoder-only representations for similarity scoring. Our experiments investigated two context enrichment strategies, evaluating the impact of replacing CPC codes with their textual descriptions. The proposed method

achieve a cross-validation Pearson correlation of 0.856, placing it competitively between standard encoder-based baselines and prompt-optimized encoder models. Our results suggest that decoder-only LLMs represent a promising direction for domain-specific technical phrase matching, with further gains likely attainable through more sophisticated prompt engineering. Future work may explore combining decoder-only representations with ensemble strategies to close the gap with state-of-the-art approaches.

References

- [Aslanyan and Wetherbee, 2022] Aslanyan, G. and Wetherbee, I. (2022). Patent phrase similarity. In *Proceedings of the SIGIR 2022 Workshop on Patent Text Mining and Semantic Technologies (PatentSemTech)*.
- [Chenghuige, 2022] Chenghuige (2022). U.S. Patent Phrase to Phrase Matching – 1st place solution. <https://github.com/chenghuige/U.S.-Patent-Phrase-to-Phrase-Matching>. Kaggle Competition Winning Solution.
- [Jeremy, 2022] Jeremy, H. (2022). U.S. Patent Phrase to Phrase Matching – most voted solution. <https://www.kaggle.com/code/jhoward/iterate-like-a-grandmasterIterate-like-a-grandmaster>. Kaggle Competition Most Voted Solution.
- [Yu et al., 2024] Yu, L., Liu, B., Lin, Q., Zhao, X., and Che, C. (2024). Semantic similarity matching for patent documents using ensemble bert-related model and novel text processing method. *arXiv:2401.06782*.